

### 1. ¿Cómo es leído el encabezado del programa ejecutable NachOS?

```
executable->ReadAt((char *)&noffH, sizeof(noffH), 0);
```

El encabezado del programa ejecutable NachOS se lee en el constructor de AddrSpace, utilizando el método ReadAt del archivo ejecutable, para esto se declara una estructura NoffHeader noffH y se ejecuta una lectura desde la posición 0 del archivo, copiando los primeros bytes del ejecutable dentro de dicha estructura, esto permite obtener la información necesaria sobre los segmentos del programa, como código, datos inicializados y datos no inicializados.

### 2. ¿De qué tamaño es el encabezado?

El tamaño del encabezado está determinado por el tamaño de la estructura NoffHeader, en el código este tamaño se obtiene mediante sizeof(noffH), lo que permite leer exactamente la cantidad de bytes correspondiente al encabezado del ejecutable NachOS.

### 3. ¿Cuál es el número mágico?

```
if ((noffH.noffMagic != NOFFMAGIC) && (WordToHost(noffH.noffMagic) == NOFFMAGIC))  
    SwapHeader(&noffH);
```

El número mágico es el valor almacenado en noffH.noffMagic, el cual se compara contra la constante NOFFMAGIC, esta verificación permite confirmar que el archivo cargado corresponde a un ejecutable NachOS válido, si el valor no coincide NachOS revisa si se trata de un problema de orden de bytes y, si corresponde aplica SwapHeader.

### 4. ¿Cómo es establecido el tamaño del proceso?

```
// how big is address space?  
size = noffH.code.size + noffH.initData.size + noffH.uninitData.size + UserStackSize; // we need to increase the size  
// to leave room for the stack  
numPages = divRoundUp(size, PageSize); identifier "PageSize" is undefined  
size = numPages * PageSize;
```

El tamaño del proceso se establece sumando el tamaño del segmento de código, el segmento de datos inicializados, el segmento de datos no inicializados y la pila de usuario. Después, ese valor se divide entre el tamaño de página para calcular cuántas páginas necesita el proceso, como el tamaño puede no ser múltiplo exacto de una página, se utiliza divRoundUp para reservar una página adicional cuando sea necesario.

### 5. ¿Cuál es el tamaño del segmento de código (T)?

```
// then, copy in the code and data segments into memory  
if (noffH.code.size > 0)  
{  
    DEBUG('a', "Initializing code segment, at 0x%x, size %d\n", noffH.code.virtualAddr, noffH.code.size);  
    LoadSegment(executable, pageTable, noffH.code.virtualAddr, noffH.code.size, noffH.code.inFileAddr);  
}
```

El tamaño del segmento de código se obtiene del campo noffH.code.size, dentro del encabezado NOFF del programa ejecutable. Este segmento contiene las instrucciones del programa de usuario que serán cargadas en memoria para su ejecución. En la función, este valor se usa para saber cuántos bytes deben leerse desde el archivo ejecutable y colocarse en memoria.

## 6. ¿Cuál es el tamaño del segmento de datos (D)?

```
}  
if (noffH.initData.size > 0)  
{  
    DEBUG('a', "Initializing data segment, at 0x%x, size %d\n", noffH.initData.virtualAddr, noffH.initData.size);  
    LoadSegment(executable, pageTable, noffH.initData.virtualAddr, noffH.initData.size, noffH.initData.inFileAddr);  
}
```

El tamaño del segmento de datos inicializados se obtiene del campo `noffH.initData.size`, este valor indica cuántos bytes de datos deben cargarse desde el archivo ejecutable hacia la memoria del proceso, estos datos corresponden a variables o información que ya tiene un valor inicial definido antes de que el programa comience a ejecutarse.

## 7. ¿Cuál es el tamaño de la pila (S)?

El tamaño de la pila está definido por la constante `UserStackSize`, esta pila se reserva como parte del espacio de direcciones del proceso, aunque no se lee desde el archivo ejecutable, su espacio se incluye en el cálculo total de memoria que necesita el programa para ejecutarse.

## 8. ¿Cuál es el tamaño de una página?

```
numPages = divRoundUp(size, PageSize);  
size = numPages * PageSize;
```

El tamaño de una página está definido por la constante `PageSize`, en esta implementación cada página tiene un tamaño de 128 bytes, este valor se usa para calcular cuántas páginas necesita un proceso para dividir las direcciones virtuales en página y offset, y para cargar los segmentos del programa por partes.

## 9. ¿Alguno de estos tamaños puede ser cero?

Sí algunos tamaños pueden ser cero, por ejemplo el segmento de datos inicializados puede tener tamaño cero si el programa no tiene variables o datos con valores iniciales, también los datos no inicializados podrían variar según el programa. Entonces antes de cargar los segmentos, el constructor de `AddrSpace` verifica si su tamaño es mayor que cero, pero la pila siempre debe reservarse como parte del espacio del proceso, aunque no se cargue desde el archivo.

## 10. ¿Cómo el programa es cargado (del disco) a memoria?

```
if (noffH.code.size > 0)  
{  
    DEBUG('a', "Initializing code segment, at 0x%x, size %d\n", noffH.code.virtualAddr, noffH.code.size);  
    LoadSegment(executable, pageTable, noffH.code.virtualAddr, noffH.code.size, noffH.code.inFileAddr);  
}
```

```

static void LoadSegment(OpenFile *executable, TranslationEntry *pageTable, int virtualAddr, int size, int inFileAddr)
{
    int remaining = size;           // Bytes que faltan por cargar
    int currentVirtualAddr = virtualAddr; // Dirección virtual actual del segmento
    int currentFileAddr = inFileAddr; // Posición actual dentro del archivo ejecutable

    while (remaining > 0)
    {
        // Determina la página virtual y el desplazamiento dentro de esa página.
        int virtualPage = currentVirtualAddr / PageSize; // identifier "PageSize" is undefined
        int offset = currentVirtualAddr % PageSize;

        // Obtiene la página física asignada a esa página virtual.
        int physicalPage = pageTable[virtualPage].physicalPage;

        // Calcula la dirección real dentro de la memoria principal.
        int physicalAddr = physicalPage * PageSize + offset;

        // Calcula cuántos bytes se pueden leer en esta página.
        int bytesToRead = PageSize - offset;

        if (bytesToRead > remaining)
        {
            bytesToRead = remaining;
        }

        // Lee del archivo ejecutable y coloca los bytes en la memoria física correcta.
        executable->ReadAt(&(machine->mainMemory[physicalAddr]), bytesToRead, currentFileAddr); // identifier "machine" is undefined

        // Avanza a la siguiente parte del segmento.
        remaining -= bytesToRead;
        currentVirtualAddr += bytesToRead;
        currentFileAddr += bytesToRead;
    }
}

```

(Función propia implementada)

El programa es cargado desde disco a memoria utilizando el archivo ejecutable recibido por el constructor de AddrSpace, primero se lee el encabezado NOFF para identificar los segmentos del programa, como código y datos inicializados, luego cada segmento se lee desde su posición correspondiente dentro del archivo y se coloca en memoria principal. En la función, esta carga se realiza con LoadSegment para ubicar los bytes en las páginas físicas asignadas por la tabla de páginas.

## 11. ¿Cómo se realiza la lectura del disco?

```

// Lee del archivo ejecutable y coloca los bytes en la memoria física correcta.
executable->ReadAt(&(machine->mainMemory[physicalAddr]), bytesToRead, currentFileAddr);

```

La lectura del disco se realiza mediante el método ReadAt, este método permite leer bytes desde una posición específica del archivo ejecutable y copiarlos hacia una ubicación en memoria, en este caso se usa primero para leer el encabezado NOFF y luego para cargar los segmentos de código y datos inicializados del programa.

## 12. Analice la clase encargada de realizar esta lectura del disco

```

AddrSpace(OpenFile *executable); // Create an address space,
// initializing it with the program
// stored in the file "executable"

```

La clase encargada de realizar la lectura del disco es OpenFile, en el constructor de AddrSpace, el programa ejecutable se recibe como un puntero OpenFile \*executable, y a través de ese objeto se llama al método ReadAt para leer el encabezado y los segmentos del programa.

### 13. ¿Cómo se determina el archivo de donde se realiza la lectura ?

```
./nachos -x ../test/halt
Machine halting!

Ticks: total 23, idle 0, system 10, user 13
Disk I/O: reads 0, writes 0
Console I/O: reads 0, writes 0
Paging: faults 0
Network I/O: packets received 0, sent 0

Cleaning up...
```

El archivo de donde se realiza la lectura se determina por el programa de usuario que se ejecuta con la opción -x, ese archivo es abierto por NachOS y luego se pasa al constructor de AddrSpace como un objeto OpenFile \*executable, a partir de ese objeto el constructor puede leer el encabezado y los segmentos del ejecutable.

En este ejemplo el archivo leído es: ../test/halt

### 14. ¿Cómo se indica la cantidad de bytes que se quiere leer?

```
executable->ReadAt((char *)&noffH, sizeof(noffH), 0);
```

La cantidad de bytes que se quiere leer se indica mediante el segundo parámetro de ReadAt, para leer el encabezado se usa sizeof(noffH), porque se necesita leer exactamente el tamaño de la estructura NoffHeader; para cargar segmentos por páginas, se usa una variable como bytesToRead, que permite leer solamente los bytes que caben en la página actual o los bytes restantes del segmento.

La estructura de ReadAt se puede ver así: ReadAt(destino, cantidadBytes, posicionArchivo);

### 15. ¿Cómo hacer para leer del archivo solo una página (128 bytes)?

```
// Lee del archivo ejecutable y coloca los bytes en la memoria física correcta.
executable->ReadAt(&(machine->mainMemory[physicalAddr]), bytesToRead, currentFileAddr);
```

Se utiliza el método ReadAt indicando como cantidad de bytes un máximo de PageSize, que corresponde a 128 bytes, en la función esto se controla con la variable bytesToRead, la cual calcula cuántos bytes caben en la página actual, si el segmento tiene menos bytes restantes que una página completa, solo se leen los bytes faltantes.

### 16. ¿Cuál es la posición dentro del archivo de lectura?

La posición dentro del archivo de lectura se indica con el tercer parámetro del método ReadAt, para el segmento de código se usa inicialmente noffH.code.inFileAddr, y para el segmento de datos inicializados se usa noffH.initData.inFileAddr. Durante la carga por páginas, esta posición se actualiza mediante currentFileAddr, avanzando conforme se leen bytes del archivo.

### 17. ¿Cómo es colocada en la memoria la información leída del disco?

```
// Determina la página virtual y el desplazamiento dentro de esa página.  
int virtualPage = currentVirtualAddr / PageSize;    identifier "PageSize"  
int offset = currentVirtualAddr % PageSize;  
  
// Obtiene la página física asignada a esa página virtual.  
int physicalPage = pageTable[virtualPage].physicalPage;  
  
// Calcula la dirección real dentro de la memoria principal.  
int physicalAddr = physicalPage * PageSize + offset;
```

La información leída del disco se coloca en machine->mainMemory, que representa la memoria principal simulada de NachOS, en la función que se agregó, antes de copiar los bytes leídos, se calcula la página virtual correspondiente, se obtiene la página física asignada en la tabla de páginas y luego se calcula la dirección física real, de esta manera el contenido del ejecutable se ubica en la posición de memoria que corresponde al proceso.

#### **18. ¿Cuál es la posición en la memoria?**

La posición en memoria está dada por la dirección física calculada en la función agregada, LoadSegment, se obtiene multiplicando la página física asignada por el tamaño de página y sumando el offset correspondiente, así se determina la posición exacta dentro de machine->mainMemory donde deben colocarse los bytes leídos desde el archivo ejecutable.

### 19. ¿Cómo se puede cambiar esta dirección?

```
for (i = 0; i < numPages; i++)  
{  
    int physicalPage = memoryMap->Find();  
  
    ASSERT(physicalPage != -1);  
  
    pageTable[i].virtualPage = i;  
    pageTable[i].physicalPage = physicalPage;  
    pageTable[i].valid = true;  
    pageTable[i].use = false;  
    pageTable[i].dirty = false;  
    pageTable[i].readOnly = false;  
}
```

Puede cambiarse modificando la página física asignada a cada página virtual en la tabla de páginas, en lugar de asumir que la página virtual y la página física tienen el mismo número, se utiliza un BitMap para encontrar una página física libre, luego esa página física se guarda en `pageTable[i].physicalPage`, entonces al final la misma dirección virtual del programa puede terminar ubicada en una dirección física diferente dentro de `mainMemory`.

### 20. Analice la estrategia que va a seguir para colocar las páginas de 128 bytes leídas del archivo ejecutables en las posiciones de memoria correspondientes (frames) también de 128 bytes

La estrategia utilizada consiste en leer los segmentos del ejecutable por bloques de hasta 128 bytes y colocarlos en los frames físicos asignados al proceso, para esto se calcula la página virtual correspondiente a cada dirección del segmento y se consulta la tabla de páginas para obtener la página física asociada. Con lo anterior, se calcula la dirección física real dentro de `machine->mainMemory` y se copian ahí los bytes leídos desde el archivo ejecutable, logra que las páginas virtuales del programa no tengan que estar ubicadas en las mismas páginas físicas.

### 21. Analice cómo va a manejar las páginas que se leen del archivo cuyo tamaño es menor que 128 bytes

```
if (bytesToRead > remaining)  
{  
    bytesToRead = remaining;  
}
```

Cuando una página no se llena completamente, solo se leen los bytes restantes del segmento, esto se controla con la variable `bytesToRead`, que evita leer más información de la necesaria, el resto de la página queda en cero porque previamente las páginas físicas asignadas al proceso fueron limpiadas con `bzero`.

## 22. Analice una estrategia para manejar páginas que pueden tener información de segmentos distintos, por ejemplo de código (T) y datos (D)

Se manejan usando el cálculo de página virtual y offset, así cada segmento se coloca exactamente en la posición que le corresponde dentro de la página física, aunque una misma página pueda contener parte de código y parte de datos.

## 23. Funcionamiento de TranslationEntry y pageTable

TranslationEntry es la estructura que utiliza NachOS para representar cada entrada de la tabla de páginas de un proceso, su función principal es relacionar una página virtual con una página física, además de almacenar información de control como si la página es válida, si fue usada, modificada o si es de solo lectura. Cuando el procesador MIPS simulado realiza una lectura o escritura de memoria, NachOS utiliza la pageTable para traducir la dirección virtual del programa a una dirección física dentro de machine->mainMemory, esta traducción se realiza tomando la dirección virtual, calculando la página virtual y el offset, y luego buscando en la tabla cuál página física corresponde.

Inicialmente, NachOS asignaba cada página virtual a la misma página física, por ejemplo 0 -> 0, 1 -> 1, 2 -> 2, pero para poder ejecutar varios programas en la misma memoria física, esta asignación debía cambiarse, por eso se incorporó el uso del BitMap, permitiendo asignar a cada página virtual cualquier página física libre disponible.

```
// First, set up the translation
this->pageTable = new TranslationEntry[numPages];    expected a type specifier
for (i = 0; i < numPages; i++)
{
    int physicalPage = memoryMap->Find();

    ASSERT(physicalPage != -1);

    pageTable[i].virtualPage = i;
    pageTable[i].physicalPage = physicalPage;
    pageTable[i].valid = true;
    pageTable[i].use = false;
    pageTable[i].dirty = false;
    pageTable[i].readOnly = false;
}
```

## 24. Uso de BitMap para administrar la memoria física

La clase BitMap se utiliza para representar el estado de las páginas físicas de la memoria simulada, cada posición del mapa de bits representa una página física: si el bit está libre, esa página puede asignarse a un proceso; si está ocupado no debe reutilizarse hasta que el proceso la libere, esto permite llevar un control más ordenado de la memoria disponible. En la entrega se crea un BitMap con el tamaño de la memoria física expresado en páginas, usando NumPhysPages, si un proceso necesita memoria se utiliza Find() para buscar una página física libre y marcarla como ocupada, cuando el proceso finaliza se utiliza Clear() para liberar las páginas que estaban asignadas a ese proceso.

```
#include "copyright.h"
#include "system.h"
#include "addrspace.h"
#include "noff.h"
#include "bitmap.h"

static BitMap *memoryMap = NULL;
```

```
if (memoryMap == NULL)
{
    memoryMap = new BitMap(NumPhysPages);    identifier
}
```

## 25. Diagrama de la estructura de los programas de NachOS

